

Kubernetes - Modul Basics

Agenda

1. Docker-Grundlagen

- [Übersicht Architektur](#)
- [Was ist ein Container ?](#)
- [Was sind container images](#)
- [Container vs. Virtuelle Maschine](#)
- [Was ist ein Dockerfile](#)
- [Dockerfile - image kleinhalten](#)

2. Kubernetes - Überblick

- [12-Factor-App - Design Prinzipien fuer Cloud Native Anwendungen](#)
- [Warum Kubernetes, was macht Kubernetes](#)
- [Aufbau Allgemein](#)
- [Kubernetes Architektur Deep-Dive](#)
- [Wann macht Kubernetes Sinn, wann nicht?](#)
- [Welches System ? \(minikube, micro8ks etc.\)](#)

3. kubectl

- [kubectl einrichten mit namespace](#)
- [kubectl cheatsheet kubernetes](#)

4. Kubernetes Praxis API-Objekte

- [Das Tool kubectl \(Devs/Ops\) - Spickzettel](#)
- [kubectl example with run](#)
- [Bauen einer Applikation mit Resource Objekten](#)
- [Anatomie einer Webanwendungen](#)
- [kubectl/manifest/pod](#)
- ReplicaSets kurz erwähnen (Deployments verwalten sie automatisch)
- Deployments (Devs/Ops)
- [kubectl/manifest/deployments](#)
- Services (Devs/Ops)
- [kubectl/manifest/service](#)

5. Kubernetes Ingress

- [Hintergrund Ingress](#)
- [Install Traefik-IngressController](#)
- [Ingress mit traefik](#)

6. Kubernetes Praxis (Stateful Sets)

- [Hintergrund statefulsets](#)
- [Example stateful set](#)

7. Kubernetes Storage, ConfigMaps & Secrets

- [Überblick Persistent Volumes \(CSI\)](#)
- [Übung Persistent Storage](#)
- [Credentials in Kubernetes verwenden - welche Möglichkeiten gibt es?](#)

- [ConfigMap Example MariaDB](#)
- [ConfigMap Example nginx.conf](#)
- [Secrets Example MariaDB](#)

8. Kubernetes API-Objekte (Teil 2)

- [Jobs](#)

9. Helm (Kubernetes Paketmanager)

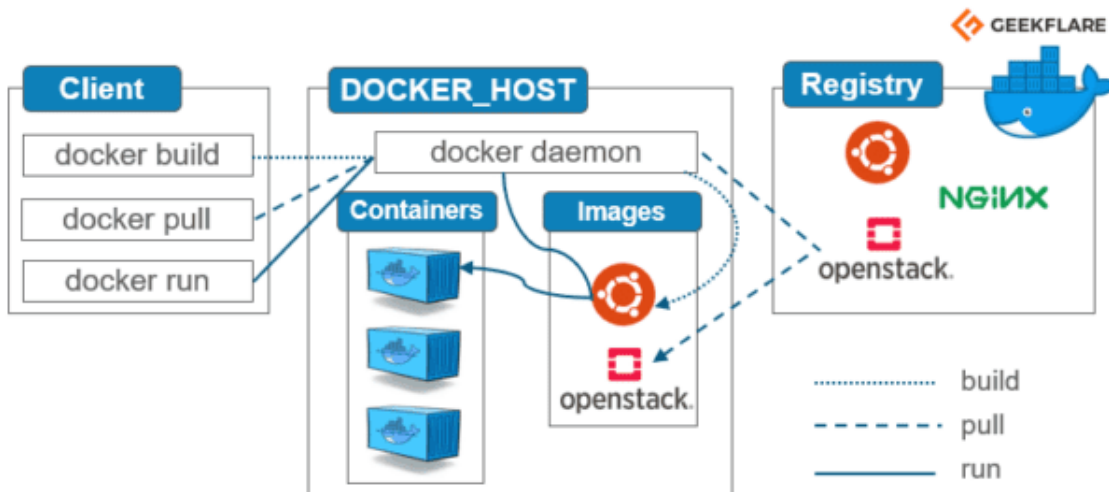
- [Helm - Was kann Helm](#)
- [Helm Grundlagen](#)
- [Helm Warum ?](#)
- [Helm Example](#)
- [Installation, Upgrade, Uninstall helm-Chart exercise - simple \(mariadb-cloudpirates\)](#)
- [Helm Spickzettel](#)

10. Kubernetes Debugging (Troubleshooting)

- [Netzwerkverbindung zum Pod testen](#)
- [Befehle in pod ausführen - Übung](#)
- [Welche Pods mit Namen gehören zu einem Service](#)
- [Übung: Service-Verbindungsprobleme debuggen mit kubectl debug \(ohne NetworkPolicy\)](#)

Docker-Grundlagen

Übersicht Architektur



Was ist ein Container ?

- vereint in sich Software
 - Bibliotheken
 - Tools
 - Konfigurationsdateien
 - keinen eigenen Kernel
 - gut zum Ausführen von Anwendungen auf verschiedenen Umgebungen
-
- Container sind entkoppelt
 - Container sind voneinander unabhängig
 - Können über wohldefinierte Kommunikationskanäle untereinander Informationen austauschen
-
- Durch Entkopplung von Containern:
 - o Unverträglichkeiten von Bibliotheken, Tools oder Datenbank können umgangen werden, wenn diese von den Applikationen in unterschiedlichen Versionen benötigt werden.

Anwendungsfälle

- Unterschiedliche Versionen einer Applikation (z.B. MariaDB-Server) auf einem Linux-System betreiben
- Gute Skalieren zu können (Beispiel: Bestellanzahl steigt (wir brauchen bei Ressourcen für Shop-Katalog und Warenkorb), aber nicht für Registrierung
 - Sprachagnostik /ein Service in python, einer in Rust

Was sind container images

- Container Image benötigt, um zur Laufzeit Container-Instanzen zu erzeugen
- Bei Docker werden Docker Images zu Docker Containern, wenn Sie auf einer Docker Engine als Prozess ausgeführt werden
- Man kann sich ein Docker Image als Kopiervorlage vorstellen.

- Diese wird genutzt, um damit einen Docker Container als Kopie zu erstellen

Container vs. Virtuelle Maschine

```
VM's virtualisieren Hardware
Container virtualisieren Betriebssystem
```

Was ist ein Dockerfile

Grundlagen

- Textdatei, die Linux - Kommandos enthält
 - die man auch auf der Kommandozeile ausführen könnte
 - Diese erledigen alle Aufgaben, die nötig sind, um ein Image zusammenzustellen
 - mit `docker build` wird dieses image erstellt

Beispiel

```
FROM node:12-alpine
RUN apk add --no-cache python2 g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
## übersetzt: node src/index.js
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Jetzt wird gebaut...

```
cd buildtest
docker build -t trainerimage:1.0 .
```

Dockerfile - image kleinhalten

- Delete all files that are not needed in image

Example

```
### Delete files needed for installation
### Right after the installation of the necessary
## Variante 2
## nano Dockerfile
FROM ubuntu:22.04
RUN apt-get update && \
    apt-get install -y inetutils-ping && \
    rm -rf /var/lib/apt/lists/*
## CMD ["/bin/bash"]
```

Example 2: Start from scratch

- <https://codeburst.io/docker-from-scratch-2a84552470c8>

Kubernetes - Überblick

12-Factor-App - Design Prinzipien fuer Cloud Native Anwendungen

- Das sind best-practices

Die 12-Factors stammen von Heroku und beschreiben, wie eine App aussehen muss, damit sie sich problemlos in einer Cloud-Plattform betreiben lässt – also genau das, was Kubernetes heute von einem Workload erwartet.

- Ursprünglich entwickelt von heroku 2011
- Ursprünglich gedacht für cloud-native apps
- Auch gut für microservices anwendbar

Anwendung

- Checkliste: Gilt das für meinen Service ?

Hier ist die Tabelle der Twelve-Factor App Principles:

#	Prinzip	Beschreibung
1	Codebase	Versionsverwaltetes Code-Repository
2	Dependencies	Abhängigkeiten sollten extern verwaltet werden. (spielte zur Zeit von heroku eine Rolle, weil Software auf dem Host ausgeführt wurde. Man soll sich also nicht darauf verlassen, was auf dem Host existiert. Bei Docker/Kubernetes ist das bereits im Container-Image selbst geregelt. Man könnte also sagen, die Regel ist bei Docker-Images ohnehin erfüllt.
3	Config	Konfiguration als Umgebungsvariablen
4	Backing Services	Datenbanken, Messaging etc. als externe Ressourcen
5	Build, Release, Run	Drei unabhängige Deployment-Schritte
6	Stateless Processes	Zustandslose, unabhängige Prozesse zum guten Skalieren
7	Port Binding	App bindet direkt an Port
8	Concurrency	Apps sollten in Module aufgeteilt werden zur einfachen Skalierung
9	Disposability	Schneller Start und einfaches Herunterfahren
10	DEV/PROD Parity	Entwicklungsumgebung und Produktion möglichst ähnlich
11	Logs	Logs als Event-Streams behandeln (wie bei docker / kubernetes)
12	Admin Processes	Admin-Aufgaben als One-off-Prozesse Einmalige Aktion möglichst gesciptet und versioniert und sie sollten in der gleichen

Umgebung umgesetzt werden.
Job/Cronjob/initContainer

Warum Kubernetes, was macht Kubernetes

Ausgangslage

- Ich habe jetzt einen Haufen Images, aber:
 - Wie bekomme ich die auf die Systeme.
 - Und wie halte ich den Verwaltungsaufwand in Grenzen.
- Lösung: Kubernetes -> ein Orchestrierungstool

Hintergründe

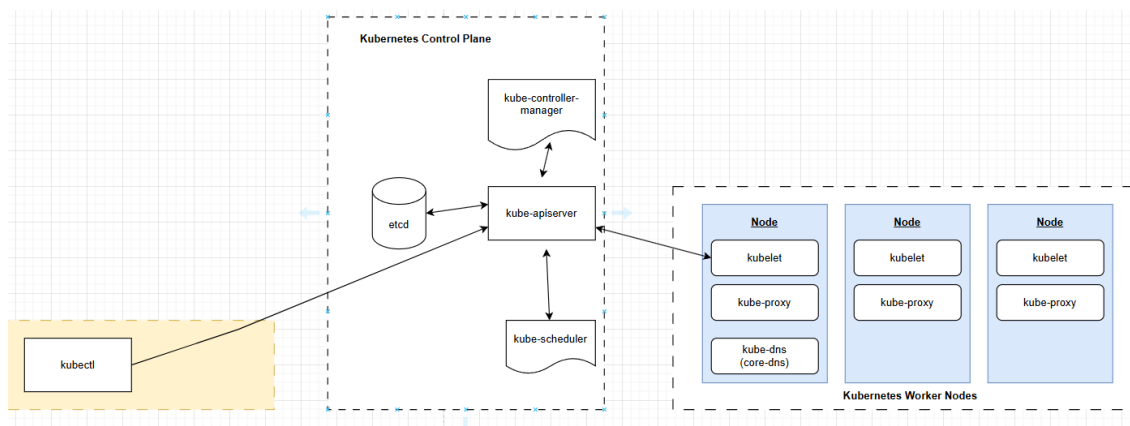
- Gegenüber Virtualisierung von Hardware - x-fache bessere Auslastung
- Google als Ausgangspunkt (Borg)
- Software 2014 als OpenSource zur Verfügung gestellt
- Optimale Ausnutzung der Hardware, hunderte bis tausende Dienste können auf einigen Maschinen laufen (Cluster)
- Immutable - System
- Selbstheilend

Wozu dient Kubernetes

- Orchestrierung von Containern
- am gebräuchlichsten aktuell Docker -Images

Aufbau Allgemein

Schaubild



Komponenten / Grundbegriffe

Control Plane (Master)

Aufgaben

- Der Control Plane (Master) koordiniert den Cluster
- Der Control Plane (Master) koordiniert alle Aktivitäten in Ihrem Cluster
 - Planen von Anwendungen
 - Verwalten des gewünschten Status der Anwendungen

- Skalieren von Anwendungen
- Rollout neuer Updates.

Komponenten des Masters

ETCD

- Verwalten der Konfiguration und des Status des Clusters (key/value - pairs)

KUBE-CONTROLLER-MANAGER

- Zuständig für die Überwachung der Stati im Cluster mit Hilfe von endlos loops.
- kommuniziert mit dem Cluster über die kubernetes-api (bereitgestellt vom kube-api-server)

KUBE-API-SERVER

- provides api-frontend for administration (no gui)
- Exposes an HTTP API (users, parts of the cluster and external components communicate with it)
- REST API

KUBE-SCHEDULER

- assigns Pods to Nodes.
- scheduler determines which Nodes are valid placements for each Pod in the scheduling queue (according to constraints and available resources)
- The scheduler then ranks each valid Node and binds the Pod to a suitable Node.
- Reference implementation (other schedulers can be used)

Nodes

- Nodes sind virtuelle oder physische Maschinen auf denen die notwendigen Kubernetes-System-Komponenten (Software)
- Worker Nodes (Knoten) sind die Arbeiter (Maschinen), die Anwendungen ausführen
- Ref: <https://kubernetes.io/de/docs/concepts/architecture/nodes/>

Pod/Pods

- Pods sind die kleinste verwaltbare Einheit, die in Kubernetes erstellt und verwaltet werden können.
- Ein Pod (übersetzt Gruppe) ist eine Gruppe von einem oder mehreren Containern
 - gemeinsam genutzter Speicher- und Netzwerkressourcen
 - Befinden sich immer auf dem gleich virtuellen Server

Node (Minion) - components

General

- On the nodes we will rollout the applications

kubelet

```
Node Agent that runs on every node (worker)
Er stellt sicher, dass Container in einem Pod ausgeführt werden.
```

Kube-proxy

- Läuft auf jedem Node
- = Netzwerk-Proxy für die Kubernetes-Netzwerk-Services.
- Kube-proxy verwaltet die Netzwerkkommunikation der Services innerhalb des Clusters

Referenzen

- <https://www.redhat.com/de/topics/containers/kubernetes-architecture>

Kubernetes Architektur Deep-Dive

- <https://github.com/jmetzger/training-kubernetes-advanced/assets/1933318/1ca0d174-f354-43b2-81cc-67af8498b56c>

Wann macht Kubernetes Sinn, wann nicht?

Wann nicht sinnvoll ?

- Anwendung, die ich nicht in Container "verpackt" habe
- Spielt der Dienstleister mit (Wartungsvertrag)
- Kosten / Nutzenverhältnis (Umstellen von Container zu teuer)
- Anwendung lässt sich nicht skalieren
 - z.B. Bottleneck Datenbank
 - Mehr Container bringen nicht mehr (des gleichen Typs)

Wo spielt Kubernetes seine Stärken aus ?

- Skalieren von Anwendungen.
- bessere Hochverfügbarkeit out-of-the-box
- Heilen von Systemen (neu starten von Containern)
- Automatische Überwachung (mit deklarativem Management) - ich beschreibe, was ich will
- Neue Versionen auszurollen (Canary Deployment, Blue/Green Deployment)

Mögliche Nachteile

- Steigert die Komplexität.
- Debugging wird u.U. schwieriger
- Mit Kubernetes erkaufe ich mir auch, die Notwendigkeit.
 - Über adequate Backup-Lösungen nachzudenken (Moving Target, Kubernetes Aware Backups)
 - Bereitsstellung von Monitoring
 - Bereitsstellung Observability (Log-Aggregierungslösung, Tracing)

Klassische Anwendungsfällen (wo Kubernetes von Vorteil)

- Webbasierte Anwendungen (z.B. auch API's bzw. Web)
- Ausser Problematik: Session StickyNess

Welches System ? (minikube, micro8ks etc.)

Überblick der Systeme

General

```
kubernetes itself has not convenient way of doing specific stuff like
creating the kubernetes cluster.
```

```
So there are other tools/distri around helping you with that.
```

Kubeadm

General

- The official CNCF (<https://www.cncf.io/>) tool for provisioning Kubernetes clusters (variety of shapes and forms (e.g. single-node, multi-node, HA, self-hosted))
- Most manual way to create and manage a cluster

microk8s

Prerequisites:

- at least 4 GB of ram per maschine

General

- Created by Canonical (Ubuntu)
- Runs on Linux
- Runs only as snap
- (In the meantime it is also available for Windows/Mac)
- HA-Cluster (control plane)

Production-Ready ?

- Short answer: YES

Quote canonical (2020):

MicroK8s is a powerful, lightweight, reliable production-ready Kubernetes distribution. It is an enterprise-grade Kubernetes distribution that has a small disk and memory footprint while offering carefully selected add-ons out-the-box, such as Istio, Knative, Grafana, Cilium and more. Whether you are running a production environment or interested in exploring K8s, MicroK8s serves your needs.

Ref: <https://ubuntu.com/blog/introduction-to-microk8s-part-1-2>

Advantages

- Easy to setup HA-Cluster (multi-node control plane)
- Easy to manage

Disadvantages

- Nicht so flexible wie kubeadm
- z.B. freie Wahl des CNI - Providers (z.B Calico)
- nicht so flexibel bei speziell config (z.B.andere IP-Ranges)

minikube

Disadvantages

- Not usable / intended for production

Advantages

- Easy to set up on local systems for testing/development (Laptop, PC)
- Multi-Node cluster is possible
- Runs und Linux/Windows/Mac
- Supports plugin (Different name ?)

k3s (wsl oder virtuelle Maschine)

- sehr schlank.
- lokal installierbar (eine node, ca 5 minuten)
- ein einziges binary
- <https://docs.k3s.io/quick-start>

kind (Kubernetes-In-Docker)

General

- Runs in docker container

For Production ?

Having a footprint, where kubernetes runs within docker and the applikations run within docker as docker containers it is not suitable for production.

kubectl

kubectl einrichten mit namespace

config einrichten

```
cd
mkdir -p .kube
cd .kube
cp /tmp/config config
ls -la
## Alternative: nano config befüllen
## das bekommt ihr aus Eurem Cluster Management Tool
```

```
kubectl cluster-info
```

Arbeitsbereich konfigurieren

```
NS=jochen # hier tragt ihr euren eigenen Namen ein z.B. NS=peter
```

```
kubectl create ns $NS
kubectl get ns
kubectl config set-context --current --namespace $NS
kubectl get pods
```

```
## Beispiel
## kubectl create ns jochen
## kubectl get ns
## kubectl config set-context --current --namespace jochen
## kubectl get pods
```

kubectl cheatsheet kubernetes

- <https://kubernetes.io/docs/reference/kubectl/cheatsheet/>

Kubernetes Praxis API-Objekte

Das Tool kubectl (Devs/Ops) - Spickzettel

Hilfe

```
## Hilfe zu befehl
kubectl help config
## Hilfe nächste Ebene
kubectl config set-context --help
```

Allgemein

```
## Zeige Informationen über das Cluster
kubectl cluster-info

## Welche Ressourcen / Objekte gibt es, z.B. Pod
kubectl api-resources
kubectl api-resources | grep namespaces

## Hilfe zu object und eigenschaften bekommen
kubectl explain pod
kubectl explain pod.metadata
kubectl explain pod.metadata.name
```

namespaces

```
kubectl get ns
kubectl get namespaces

## namespace wechseln, z.B. nach Ingress
kubectl config set-context --current --namespace=ingress
## jetzt werden alle Objekte im Namespace Ingress angezeigt
kubectl get all,configmaps

## wieder zurückwechseln.
## der standardmäßige Namespace ist 'default'
kubectl config set-context --current --namespace=default
```

Arbeiten mit manifesten

```
kubectl apply -f nginx-replicaset.yml
## Wie ist aktuell die hinterlegte config im system
kubectl get -o yaml -f nginx-replicaset.yml

## Änderung in nginx-replicaset.yml z.B. replicas: 4
## dry-run - was wird geändert
kubectl diff -f nginx-replicaset.yml

## anwenden
kubectl apply -f nginx-replicaset.yml

## Alle Objekte aus manifest löschen
```

```
kubectl delete -f nginx-replicaset.yml

## Recursive Löschen
cd ~/manifests
## multiple subfolders subfolders present
kubectl delete -f . -R
```

Ausgabeformate / Spezielle Informationen

```
## Ausgabe kann in verschiedenen Formaten erfolgen
kubectl get pods -o wide # weitere Informationen
## im json format
kubectl get pods -o json

## gilt natürlich auch für andere kommandos
kubectl get deploy -o json
kubectl get deploy -o yaml

## Label anzeigen
kubectl get deploy --show-labels
```

Zu den Pods

```
## Start einen pod // BESSER: direkt manifest verwenden
## kubectl run podname image=imagename
kubectl run nginx image=nginx

## Pods anzeigen
kubectl get pods
kubectl get pod

## Pods in allen namespaces anzeigen
kubectl get pods -A

## Format weitere Information
kubectl get pod -o wide
## Zeige labels der Pods
kubectl get pods --show-labels

## Zeige pods mit einem bestimmten label
kubectl get pods -l app=nginx

## Status eines Pods anzeigen
kubectl describe pod nginx

## Pod löschen
kubectl delete pod nginx
## Löscht alle Pods im eigenen Namespace bzw. Default
kubectl delete pods --all
```

```
## Kommando in pod ausführen
kubectl exec -it nginx -- bash
```

Deployments

```
kubectl -n ingress rollout restart deployment traefik
ik
```

Alle Objekte anzeigen

```
## Nur die wichtigsten Objekte werden mit all angezeigt
kubectl get all
## Dies, kann ich wie folgt um weitere ergänzen
kubectl get all,configmaps
## Über alle Namespaces hinweg
kubectl get all -A
```

Logs

```
kubectl logs <container>
kubectl logs <deployment>
## e.g.
## kubectl logs -n namespace8 deploy/nginx
## with timestamp
kubectl logs --timestamps -n namespace8 deploy/nginx
## continuously show output
kubectl logs -f <pod>
## letzten x Zeilen anschauen aus log anschauen
kubectl logs --tail=5 <your pod>
```

CRD

```
kubectl get crd
```

Referenz

- <https://kubernetes.io/de/docs/reference/kubectl/cheatsheet/>

kubectl example with run

Example (that does work)

```
## Synopsis (most simplistic example
## kubectl run NAME --image=IMAGE_EG_FROM_DOCKER
## example
kubectl run nginx --image=nginx:1.23
kubectl get pods
```

```
## on which node does it run ?
kubectl get pods -o wide
```

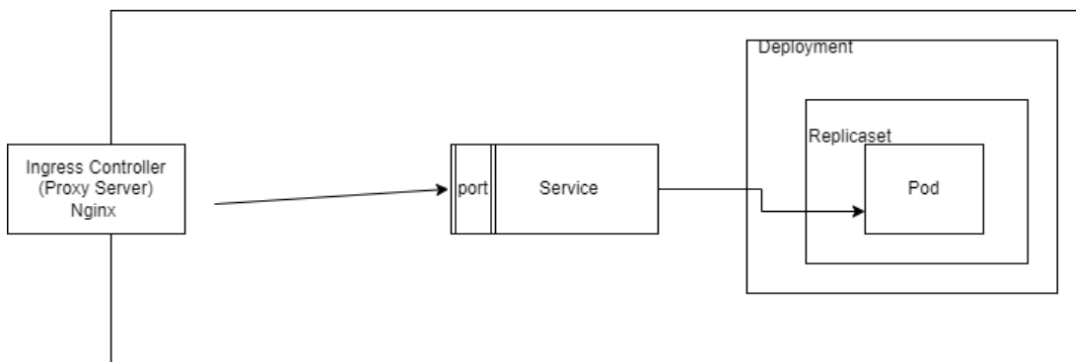
Example (that does not work)

```
kubectl run testpod --image=foo2
## ErrImagePull - Image konnte nicht geladen werden
kubectl get pods
## Weitere status - info
kubectl describe pods testpod
```

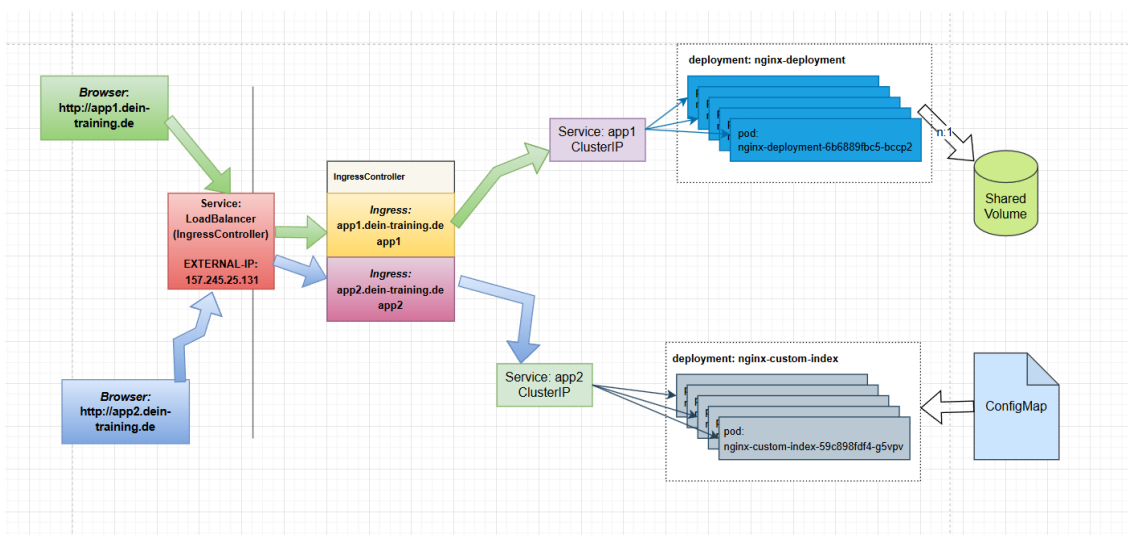
Ref:

- <https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#run>

Bauen einer Applikation mit Resource Objekten



Anatomie einer Webanwendungen



kubectl/manifest/pod

Walkthrough

```
cd
mkdir -p manifests
cd manifests/
mkdir -p 01-web
cd 01-web
nano nginx-static.yml
```

```
## vi nginx-static.yml

apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-web
  labels:
    webserver: nginx
spec:
  containers:
    - name: web
      image: nginx:1.23
```

```
kubectl apply -f nginx-static.yml
```

```
kubectl get pod/nginx-static-web -o wide
kubectl describe pod nginx-static-web
## show config
kubectl get pod/nginx-static-web -o yaml
## seitenweise anzeigen
kubectl get pod/nginx-static-web -o yaml | less
```

Aufräumen

```
kubectl delete -f .
```

kubectl/manifest/deployments

Prepare

```
cd
mkdir -p manifests
cd manifests
mkdir 03-deploy
cd 03-deploy
nano nginx-deployment.yml
```

```
## vi nginx-deployment.yml
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 8 # tells deployment to run 8 pods matching the template
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginxinc/nginx-unprivileged:1.28
        ports:
        - containerPort: 8080
```

```
kubectl apply -f .
```

Explore

```
kubectl get all
```

Optional: Change image - Version

```
nano nginx-deployment.yml
```

Version 1: (optical nicer)

```
## Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
## danach
kubectl apply -f . && watch kubectl get pods
```

Version 2:

```
## Ändern des images von nginxinc/nginx-unprivileged:1.28 -> auf 1.29
## danach
kubectl apply -f . && kubectl get all && kubectl get pods -w
```

Version 3 mit Deployment

```
## vi nginx-deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-nfs
spec:
```



```

tln1@client:~/manifests/04-service$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
NOMINATED NODE   READINESS GATES
web-nginx-754794c6d6-8pj99         1/1     Running   0           8s    10.108.1.42   node-p
8s68 <none>                        <none>
web-nginx-754794c6d6-swckk         1/1     Running   0          10s    10.108.0.121  node-p
8s68 <none>                        <none>

```

Example I : Service with ClusterIP

Schritt 1: Vorbereitung

```

cd
mkdir -p manifests
cd manifests
mkdir 04-service
cd 04-service

```

Schritt 2: Deployment erstellen

```

nano deploy.yml

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
spec:
  selector:
    matchLabels:
      web: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        web: my-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx:1.27
          ports:
            - containerPort: 80

```

```

nano service.yml

```

```

apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
spec:

```

```
type: ClusterIP
ports:
- port: 80
  protocol: TCP
selector:
  web: my-nginx
```

```
kubectl apply -f .
## wie ist die ClusterIP ?
kubectl get all
kubectl get svc svc-nginx
## Find endpoints / did svc find pods ?
kubectl describe svc svc-nginx
```

Schritt 3: Deployment löschen

```
kubectl delete -f deploy.yml
## Keine endpunkte mehr
kubectl describe svc svc-nginx
```

Schritt 4: Deployment wieder erstellen

```
kubectl apply -f .
## Endpunkte wieder da
kubectl describe svc svc-nginx
```

Example II : Short version (NodePort)

```
## Wo sind wir ?
## cd; cd manifests/04-service
```

```
nano service.yml
## in Zeile type:
## ClusterIP ersetzt durch NodePort

kubectl apply -f .
## NodePort ab 30.000 ausfindig machen
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
<u>svc-nginx</u>	NodePort	10.109.24.227	<none>	80:32708/TCP	18h

```
kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
node-p8s68	Ready	<none>	25h	v1.33.1	10.135.0.72	139.59.128.36	Debian GNU/Linux 12 (bookworm)
node-p8s6c	Ready	<none>	25h	v1.33.1	10.135.0.73	64.226.125.3	Debian GNU/Linux 12 (bookworm)
node-p8s6u	Ready	<none>	25h	v1.33.1	10.135.0.74	159.223.24.231	Debian GNU/Linux 12 (bookworm)

```
## im client Externe NodeIP und NodePort verwenden
curl http://164.92.193.245:32708
```

Example II : Service with NodePort (long version)

```
nano deploy.yml
```

```
## you will get port opened on every node in the range 30000+
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
spec:
  selector:
    matchLabels:
      web: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        web: my-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx
          ports:
            - containerPort: 80
```

```
nano service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
  labels:
    run: svc-my-nginx
spec:
  type: NodePort
  ports:
```

```
- port: 80
  protocol: TCP
selector:
  web: my-nginx
```

```
kubectl apply -f .
```

Example III: Service mit LoadBalancer (ExternalIP)

```
cd; cd manifests/04-service
nano service.yml
## in Zeile type:
## NodePort ersetzt durch LoadBalancer

kubectl apply -f .
kubectl get svc svc-nginx

kubectl describe svc svc-nginx
## hier heisst das nicht External-IP ->
## sondern
```

```
-----
LoadBalancer Ingress:      129.212.254.6 (VIP)
Port:                    <unset> 80/TCP
```

```
kubectl get svc svc-nginx -w
## spätestens nach 5 Minuten bekommen wir eine externe ip
## z.B. 41.32.44.45

curl http://41.32.44.45
```

Example getting a specific ip from loadbalancer (if supported)

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nginx2
spec:
  type: LoadBalancer
  # this line to get a specific ip if supported
  loadBalancerIP: 10.34.12.34
  ports:
    - port: 80
      protocol: TCP
  selector:
    web: my-nginx
```

Ref.

- <https://kubernetes.io/docs/concepts/services-networking/connect-applications-service/>

Kubernetes Ingress

Hintergrund Ingress

Ref. / Dokumentation

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

Install Traefik-IngressController

```
helm repo add traefik https://traefik.github.io/charts

helm upgrade -n ingress --install traefik traefik/traefik --version 40.3.0 --create-namespace --skip-crds --reset-values

kubectl -n ingress get pods
kubectl -n ingress get svc
helm -n ingress status traefik

## Use special crds helm chart instead, because it does not deploy crds for gateway-api by default
## We get an error on digitalocean doks
## Das funktioniert in neuen version nicht mehr weil last-applied zu gross waere
## helm -n ingress upgrade --install traefik-crds traefik/traefik-crds --version 1.18.0 --reset-values
## Workaround
helm template traefik-crds traefik/traefik-crds | kubectl -n ingress apply --server-side -f -
```

Ingress mit traefik

Step 1: Walkthrough

```
cd
mkdir -p manifests
cd manifests
mkdir abi
cd abi
```

```
nano apple-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apple-app
  labels:
    app: apple
spec:
```

```
replicas: 1
selector:
  matchLabels:
    app: apple
template:
  metadata:
    labels:
      app: apple
  spec:
    containers:
      - name: web
        image: hashicorp/http-echo
        args:
          - "-text=apple-<euer-name>"
```

```
nano apple-svc.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: apple-service
spec:
  type: ClusterIP
  selector:
    app: apple
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

```
nano banana-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: banana-app
  labels:
    app: banana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: banana
  template:
    metadata:
      labels:
        app: banana
    spec:
```

```
containers:
  - name: web
    image: hashicorp/http-echo
    args:
      - "-text=banana-<euer-name>"
```

```
nano banana-svc.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: banana-service
spec:
  type: ClusterIP
  selector:
    app: banana
  ports:
    - port: 80
      targetPort: 5678 # Default port for image
```

```
kubectl apply -f .
```

Step 2: Testing connection by podIP and Service

```
kubectl get svc
kubectl get pods -o wide
kubectl run podtest --rm -it --image busybox
```

```
/ # wget -O - http://<pod-ip>:5678
/ # wget -O - http://<cluster-ip>
```

Step 3: Walkthrough

```
nano ingress.yml
```

```
## Ingress
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: example-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: "<euername>.appv2.do.t3isp.de"
      http:
        paths:
          - path: /apple
            backend:
              serviceName: apple-service
```

```
        servicePort: 80
    - path: /banana
      backend:
        serviceName: banana-service
        servicePort: 80
```

```
## ingress
kubectl apply -f ingress.yml
```

Reference

- <https://matthewpalmer.net/kubernetes-app-developer/articles/kubernetes-ingress-guide-nginx-example.html>

Step 4: Find the problem

Fix 4.1: Fehler: no matches kind "Ingress" in version "extensions/v1beta1"

```
## Gibt es diese Landkarte überhaupt
kubectl api-versions
## auf welcher Landkarte/Gruppe befindet sich Ingress jetzt
kubectl explain ingress | head
## -> jetzt auf networking.k8s.io/v1
```

```
nano ingress.yml
```

```
## auf apiVersion: extensions/v1beta1
## wird -> networking.k8s.io/v1
```

```
kubectl apply -f .
```

Fix 4.2: Bad Request unknown field ServiceName / ServicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
```

```
nano ingress.yml
```

```
## Wir ersetzen
## serviceName: apple-service
## durch:
## service:
##   name: apple-service

## das gleiche für banana
```

```
kubectl apply -f .
```

Fix 4.3. BadRequest unknown field servicePort

```
## was geht für die Property backend
kubectl explain ingress.spec.rules.http.paths.backend
## und was geht für service
kubectl explain ingress.spec.rules.http.paths.backend.service
## number
kubectl explain ingress.spec.rules.http.paths.backend.service.port
```

```
## neue Variante sieht so aus
backend:
  service:
    name: apple-service
    port:
      number: 80
## das gleich für banana-service
```

```
kubectl apply -f .
```

Fix 4.4. pathType must be specified

```
## Was macht das ?
kubectl explain ingress.spec.rules.http.paths.pathType
```

```
paths:
- path: /apple
  pathType: Prefix
  backend:
    service:
      name: apple-service
      port:
        number: 80
- path: /banana
  pathType: Exact
  backend:
    service:
      name: banana-service
      port:
        number: 80
```

```
kubectl apply -f .
kubectl get ingress example-ingress
```

Step 5: bereits fertige Lösung

```
nano ingress.yml
```

```
## Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```

  name: example-ingress
spec:
  ingressClassName: traefik
  rules:
  - host: "<euername>.appv2.do.t3isp.de"
    http:
      paths:
      - path: /apple
        pathType: Prefix
        backend:
          service:
            name: apple-service
            port:
              number: 80
      - path: /banana
        pathType: Exact
        backend:
          service:
            name: banana-service
            port:
              number: 80

```

```

## ingress
kubectl apply -f ingress.yml
kubectl describe ingress

```

Step 6: Testing

```

## mit describe herausfinden, ob er die services gefunden hat
kubectl describe ingress example-ingress

```

```

## Im Browser auf:
## hier euer Name
http://jochen.appv2.do.t3isp.de/apple
http://jochen.appv2.do.t3isp.de/apple/
http://jochen.appv2.do.t3isp.de/apple/foo
http://jochen.appv2.do.t3isp.de/banana
## geht nicht
http://jochen.appv2.do.t3isp.de/banana/nix

```

Kubernetes Praxis (Stateful Sets)

Hintergrund statefulsets

Why ?

- stable network identities (always the same name across restarts) in contrast to deployments

```

Name:      web-0.nginx
Address 1: 10.244.1.6

```

```
Name:      web-1.nginx
Address 1: 10.244.2.20
```

The Pods' ordinals, hostnames, SRV records, and A record names have not changed, but the IP addresses associated with the Pods may have changed.

Features

- Scaling Up: Ordered creation on scaling (web 2 till ready then web-3 till ready and so on)

```
StatefulSet controller created each Pod sequentially
with respect to its ordinal index,

and it waited for each Pod's predecessor to be Running and Ready

before launching the subsequent Pod
```

- Scaling Down: last created pod is torn down firstly, till finished, then the one before

```
The controller deleted one Pod at a time,
in reverse order with respect to its ordinal index,
and it waited for each to be completely shutdown before deleting the next.
```

- VolumeClaimTemplate (In addition if the pod is scaled the copies will have their own storage)
 - Plus: When you delete it, it gets recreated and claims the same persistentVolumeClaim

```
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
```

- Update Strategy: RollingUpdate / OnDelete
- Feature: Staging an Update with Partitions
 - <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/#staging-an-update>
- Feature: Rolling out a canary
 - <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/#rolling-out-a-canary>

Reference

- <https://kubernetes.io/docs/concepts/workloads/controllers/statefulsets/>

Example stateful set

Schritt 1:

```
cd
mkdir -p manifests
cd manifests
mkdir sts
cd sts
```

```
nano 01-svc.yml
```

```
## vi 01-svc.yml
## Headless Service - no ClusterIP
## Just used for name resolution of pods
## web-0.nginx
## web-1.nginx
## nslookup web-0.nginx
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: ClusterIP
  ports:
    - port: 80
      name: web
  clusterIP: None
  selector:
    app: nginx
```

```
nano 02-sts.yml
```

```
## vi 02-sts.yml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  ## name des statefulset wird nachher für den dns-namen verwendet
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: registry.k8s.io/nginx-slim:0.24
          ports:
```

```
- containerPort: 80
  name: web-nginx
```

```
kubectl apply -f .
```

Schritt 2: Auflösung Namen.

```
kubectl run --rm -it podtester --image=busybox
```

```
## In der shell
## web ist der name des statefulsets
ping web-0.nginx
ping web-1.nginx
exit
```

```
## web-0 / web-1
kubectl get pods -o wide
kubectl get sts web
kubectl delete sts web
kubectl apply -f .
kubectl run --rm -it podtest --image=busybox
```

```
## in the shell
## gleicher namer, aber andere IP als beim letzten Ping
ping web-0.nginx
exit
```

```
kubectl describe svc nginx
```

Schritt 3: Aufräumen

```
kubectl delete -f .
```

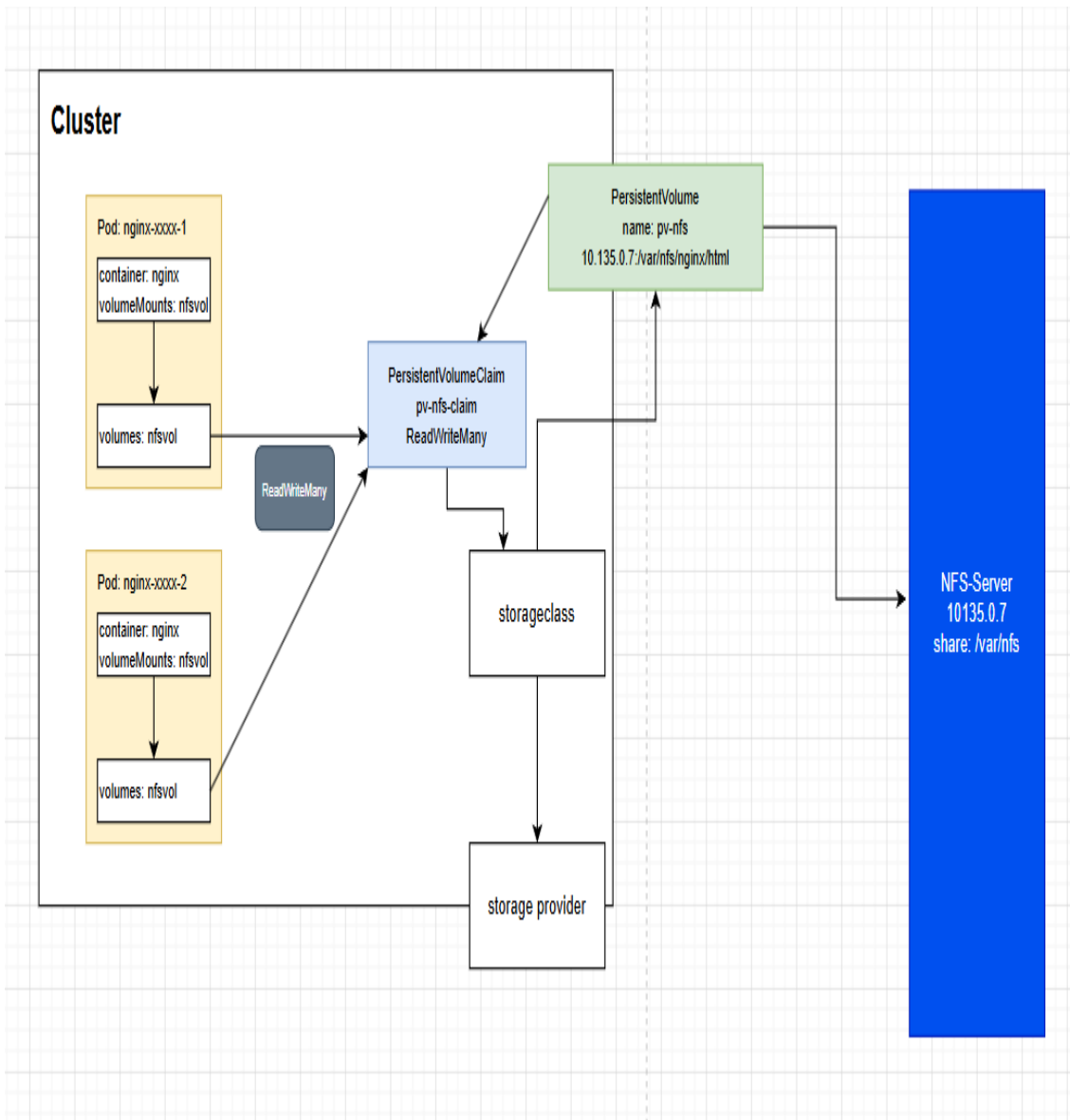
Referenz

- <https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/>

Kubernetes Storage, ConfigMaps & Secrets

Überblick Persistent Volumes (CSI)

Grafik



Überblick

Warum CSI ?

- Each vendor can create his own driver for his storage

Vorteile ?

- I. Automatically create storage when required.
- II. Make storage available to containers wherever they're scheduled.
- III. Automatically delete the storage when no longer needed.

Wie war es vorher ?

Vendor needed to wait till his code was checked in in tree of kubernetes (in-tree)

Unterschied static vs. dynamisch

The main difference relies on the moment when you want to configure storage. For instance, if you need to pre-populate data in a volume, you choose static provisioning. Whereas, if you need to create volumes on demand, you go for dynamic provisioning.

Komponenten

Treiber

- Für jede Storage Class (Storage Provider) muss es einen Treiber geben

Storage Class

Übung Persistent Storage

- Step 1 + 2 : nur Trainer
- ab Step 3: Trainees

Requirements:

- Ein NFS-Server oder eine Storage mit NFS muss im Netz zur Verfügung stehen.

Step 1: Do the same with helm - chart

```
helm repo add csi-driver-nfs https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/master/charts
helm upgrade --install csi-driver-nfs csi-driver-nfs/csi-driver-nfs --namespace kube-system --version 4.13.2 --reset-values
```

Step 2: Storage Class

```
cd
mkdir -p manifests
cd manifests
mkdir csi-storage
cd csi-storage
nano 01-storageclass.yml
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: nfs-csi
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.135.0.5
  share: /var/nfs
reclaimPolicy: Retain
volumeBindingMode: Immediate
```

```
kubectl apply -f .
```

Step 3: Persistent Volume Claim

```
cd
mkdir -p manifests
cd manifests
mkdir csi
cd csi
nano 02-pvc.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-nfs-dynamic
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
  storageClassName: nfs-csi
```

```
kubectl apply -f .
kubectl get pvc
##
kubectl get pv
```

Step 4: Pod

```
nano 03-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-nfs
spec:
  containers:
    - image: nginx:1.23
      name: nginx-nfs
      command:
        - "/bin/bash"
        - "-c"
        - set -euo pipefail; while true; do echo $(date) >> /mnt/nfs/outfile; sleep 1;
done
      volumeMounts:
        - name: persistent-storage
          mountPath: "/mnt/nfs"
          readOnly: false
  volumes:
    - name: persistent-storage
```

```
persistentVolumeClaim:
  claimName: pvc-nfs-dynamic
```

```
kubectl apply -f .
kubectl get pods
kubectl describe pods nginx-nfs
```

Step 5: Testing

```
kubectl exec -it nginx-nfs -- bash
```

```
cd /mnt/nfs
ls -la
## outfile
head /mnt/nfs/outfile
tail -f /mnt/nfs/outfile
```

```
CTRL+C
exit
```

Step 6: Destroy

```
kubectl delete -f 03-pod.yaml

### Verify in nfs - trainer !!
```

Step 7: Recreate

```
kubectl apply -f 03-pod.yaml
```

```
kubectl exec -it nginx-nfs -- bash
```

```
## is old data here ?
head /mnt/nfs/outfile
##
tail -f /mnt/nfs/outfile
```

```
CTRL + C
exit
```

Step 8: Cleanup

```
kubectl delete -f .
```

Reference:

- <https://rudimartinsen.com/2024/01/09/nfs-csi-driver-kubernetes/>

Credentials in Kubernetes verwenden - welche Möglichkeiten gibt es?

Ein Passwort, API-Key oder Token muss irgendwie in den Container. Kubernetes bietet dafür mehrere Wege — nicht alle sind gleich sicher. Diese Seite gibt den groben Überblick, bevor es an die Details (Secret-Typen, Sealed Secrets, SOPS, Vault) geht.

1. Die vier Grundwege

 Wege für Credentials in einen Container

Weg	Beispiel	Wann sinnvoll
env mit festem value	value: "s3cret"	Nie für echte Secrets — landet im Manifest/Git
env mit valueFrom.secretKeyRef	einzelner Key aus einem Secret als eine ENV-Variable	Wenn nur 1-2 Variablen gebraucht werden
envFrom.secretRef	alle Keys eines Secrets werden zu ENV-Variablen	Viele Variablen auf einmal (siehe Beispiel)
volumeMounts (Secret als Datei)	Secret wird unter /etc/secret/... gemountet	Sicherer, wenn die App auch Dateien lesen kann

Praktisches Beispiel für `valueFrom.secretKeyRef` und `envFrom.secretRef`: siehe [Übung: ENV-Variablen aus Secrets](#).

2. ENV-Variable vs. Datei (Volume Mount) — der Sicherheitsunterschied

ENV-Variablen sind bequem, aber sie "kleben" am Prozess: jeder Sub-Prozess erbt sie, und sie sind leicht auslesbar. Ein Secret als Volume-Mount ist die etwas sicherere Wahl.

 ENV-Variable vs. Volume Mount

3. Und wo kommt das Secret-Objekt selbst her?

Beide Wege (ENV oder Volume) setzen voraus, dass es bereits ein Kubernetes- `Secret` -Objekt gibt. Wie dieses sicher **erzeugt und verwaltet** wird, ist eine eigene Frage:

- [Kubernetes Secret-Typen](#) — was ein natives `Secret` -Objekt überhaupt ist (nur base64, nicht verschlüsselt!)
- [Sealed Secrets \(Bitnami\)](#) — Secret verschlüsselt in Git ablegen, Controller entschlüsselt im Cluster
- [SOPS + Age/KMS](#) — Secret-Datei lokal/CI entschlüsseln, dann `kubectl apply`
- [HashiCorp Vault](#) — zentrales Secret-Management, Injection direkt in den Pod (ganz ohne natives `Secret` -Objekt möglich)
- [Vergleich der Ansätze](#) — GitLab CI/CD vs. SOPS vs. Vault
- **AWS Secrets Manager + KMS** — siehe Schaubild unten

4. AWS Secrets Manager + KMS an Kubernetes anbinden

Wenn die Secrets bereits in AWS Secrets Manager liegen (dort per KMS verschlüsselt), ist der gängige Weg der **External Secrets Operator (ESO)**: er läuft im Cluster, holt sich über eine eng begrenzte IAM-Rolle (IRSA) periodisch

den aktuellen Wert aus Secrets Manager und legt daraus ein ganz normales Kubernetes- `Secret` an — das dann wie in Abschnitt 1 per `env / envFrom / Volume` genutzt wird.



Warum ESO die bevorzugte Wahl ist:

Kriterium	External Secrets Operator (ESO)	AWS Secrets Store CSI Driver
Verbreitung / Doku	Sehr weit verbreitet, viele Backends (nicht nur AWS)	AWS-spezifisch, weniger verbreitet
GitOps-fähig	Ja — Manifest referenziert nur die ARN	Ja — ähnliches Prinzip
Erzeugt natives <code>Secret</code> -Objekt	Ja → funktioniert mit <code>env/envFrom</code>	Optional (Sync-Feature), Standard ist reiner Volume-Mount
Ohne persistentes <code>Secret</code> -Objekt	Nein, per Design	Ja — etwas kleinere Angriffsfläche

Für die meisten Fälle (v.a. wenn ENV-Variablen gebraucht werden) ist ESO der pragmatischste Weg. Nur wenn bewusst **kein** Kubernetes- `Secret` -Objekt im Cluster persistiert werden soll, lohnt sich der CSI Driver.

Konkretes Setup Schritt für Schritt (Helm-Installation, IAM-Rolle, `SecretStore` , `ExternalSecret`): [External Secrets Operator mit AWS Secrets Manager + KMS einrichten](#).

Kurz zusammengefasst

Frage	Antwort
Darf ein Secret-Wert im Manifest stehen (<code>value: "..."</code>)?	Nein — landet im Klartext in Git/kubect!-Historie
Ist ein Kubernetes- <code>Secret</code> an sich schon "sicher"?	Nein — nur base64-kodiert, nicht verschlüsselt
ENV-Variable oder Volume-Mount?	Volume-Mount ist sicherer (kein Leak via <code>env//proc</code>)
Wie bekomme ich das Secret sicher ins Cluster?	Sealed Secrets, SOPS oder Vault — je nach Anforderung

ConfigMap Example MariaDB

Schritt 1: configmap

```
cd
mkdir -p manifests
cd manifests
mkdir cftest
cd cftest
nano 01-configmap.yml
```

```
### 01-configmap.yml
kind: ConfigMap
apiVersion: v1
metadata:
  name: mariadb-configmap
data:
  # als Wertepaare
  MARIADB_ROOT_PASSWORD: 11abc432
  TEST_CASE: "47"
```

```
kubectl apply -f .
kubectl describe cm mariadb-configmap
kubectl get cm
kubectl get cm mariadb-configmap -o yaml
```

Schritt 2: Deployment

```
nano 02-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:10.11
          envFrom:
            - configMapRef:
                name: mariadb-configmap
```

```
kubectl apply -f .
kubectl get pods
kubectl exec -it deploy/mariadb-deployment -- bash
```

```
env
env | grep ROOT
env | grep TEST
exit
```

Schritt 3: Service for mariadb

```
nano 03-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: mariadb
spec:
  type: ClusterIP
  ports:
    - port: 3306
      protocol: TCP
  selector:
    app: mariadb
```

```
kubectl apply -f 03-service.yml
```

Schritt 4: client aufsetzen

```
nano 04-client.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-client
spec:
  selector:
    matchLabels:
      app: ubuntu
  replicas: 1 # tells deployment to run 2 pods matching the template
  template: # create pods using pod definition in this template
    metadata:
      labels:
        app: ubuntu
    spec:
      containers:
        - name: service
          image: ubuntu
          command: [ "/bin/sh" , "-c", "tail -f /dev/null" ]
          envFrom:
            - configMapRef:
                name: mariadb-configmap
```

```
kubectl apply -f 04-client.yml
```

```
## im client
kubectl exec -it deploy/mariadb-client -- bash
apt update; apt install -y mariadb-client iputils-ping
```

Schritt 5: mysql-zugang von aussen erstellen

```
kubectl exec -it deploy/mariadb-deployment -- bash
```

```
mysql -uroot -p$MARIADB_ROOT_PASSWORD
```

```
## innerhalb von mysql
create user ext@'%' identified by '11abc432';
grant all on *.* to ext@'%';
```

Schritt 6: mysql von client aus testen

```
kubectl exec -it deploy/mariadb-client -- bash
```

```
mysql -uext -p$MARIADB_ROOT_PASSWORD -h mariadb
```

```
show databases;
```

Important Sidenode

- If configmap changes, deployment does not know
- So kubectl apply -f deploy.yml will not have any effect
- to fix, use stakater/reloader: <https://github.com/stakater/Reloader>

ConfigMap Example nginx.conf

Schritt 1: configmap

```
cd
mkdir -p manifests
cd manifests
mkdir nginx-conf
cd nginx-conf
nano 01-configmap.yml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  nginx.conf: |
    events {
      worker_connections 1024;
    }
    http {
      server {
        listen 80;
        location / {
          return 200 "Hello from ConfigMap!\n";
          add_header Content-Type text/plain;
```

```
    }
    location /health {
        return 200 "OK";
    }
}
}
```

```
kubect1 apply -f .
kubect1 describe cm  nginx-config
kubect1 get cm
kubect1 get cm nginx-config -o yaml
```

Schritt 2: Pod

```
nano 02-pod.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: custom-nginx
spec:
  containers:
    - name: nginx
      image: nginx:alpine
      volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf
  volumes:
    - name: nginx-config
      configMap:
        name: nginx-config
```

```
kubect1 apply -f .
kubect1 get pods
kubect1 exec -it custom-nginx -- sh
```

```
cd /etc/nginx
cat nginx.conf
exit
```

Schritt 3: busybox connection

```
## wir brauchen die pod-ip
kubect1 get pods custom-nginx -o wide
kubect1 run -it --rm podtest --image=busybox
```

```
## in der shell
wget -O - <ip-von-oben-aus-schritt-3>
```

```
exit
```

Secrets Example MariaDB

Schritt 1: secret

```
cd
mkdir -p manifests
cd manifests
mkdir secrettest
cd secrettest
```

```
kubectl create secret generic mariadb-secret --from-
literal=MARIADB_ROOT_PASSWORD=11abc432 --dry-run=client -o yaml > 01-secrets.yaml
```

```
kubectl apply -f .
kubectl get secrets
kubectl get secrets mariadb-secret -o yaml
```

Schritt 2: Deployment

```
nano 02-deploy.yml
```

```
##deploy.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  selector:
    matchLabels:
      app: mariadb
  replicas: 1
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb-cont
          image: mariadb:latest
          envFrom:
            - secretRef:
                name: mariadb-secret
```

```
kubectl apply -f .
```

Testing

```
## Führt den Befehl env in einem Pod des Deployments aus
kubectl exec deployment/mariadb-deployment -- env
## eigentlich macht er das:
## kubectl exec mariadb-deployment-c6df6f959-q6swp -- env
```

Important Sidenode

- If configmap changes, deployment does not know
- So kubectl apply -f deploy.yml will not have any effect
- to fix, use stakater/reloader: [Stakater reloader](#)

Kubernetes API-Objekte (Teil 2)

Helm (Kubernetes Paketmanager)

Helm - Was kann Helm

- **Installieren** und **Deinstallieren** von Anwendungen in Kubernetes (`helm install` / `helm uninstall`)
- **Upgraden** von bestehenden Installationen (`helm upgrade`)
- **Rollbacks** durchführen, falls etwas schief läuft (`helm rollback`)
- **Anpassen** von Anwendungen durch Konfigurationswerte (`values.yaml`)
- **Veröffentlichen** eigener Charts (z. B. in einem Helm-Repository)

Helm Grundlagen

Wo kann ich Helm-Charts suchen ?

- Im Telefonbuch von helm <https://artifacthub.io/>

Komponenten

Chart

- beinhaltet Beschreibung und Komponenten

Chart - Bereitstellungsformen

- url
- .tgz (abkürzung tar.gz) - Format
- oder Verzeichnis

```
Wenn wir ein Chart installieren, wird eine Release erstellen
(parallel: image -> container, analog: chart -> release)
```

Installation

Was brauchen wir ?

- helm client muss installiert sein

Und sonst so ?

```
## Beispiel ubuntu
## snap install --classic helm

## Cluster auf das ich zugreifen kann und im client -> helm und kubectl
```

```
## Voraussetzung auf dem Client-Rechner (helm ist nichts als anderes als ein Client-
Programm)
Ein lauffähiges kubectl auf dem lokalen System (welches sich mit dem Cluster
verbinden.
-> saubere -> .kube/config

## Test
kubectl cluster-info
```

Helm Warum ?

Ein Paket für alle Komponenten
Einfaches Installieren, Updaten und deinstallieren
Konfigurations-Values-Files übergeben zum Konfigurieren
Feststehende Struktur
Versionierung (jedes Chart hat ein Version)
In meinem Kubernetes-Cluster kann ich sehen, welche Version des Charts/der Charts
installiert wurde
Ein Chart für viele Kunden und für viele Umgebungen (Chart und passe das mit
Konfigurationswerten an)

Helm Example

Prerequisites

- helm needs a config-file (kubeconfig) to know how to connect and credentials in there
- Good: helm (as well as kubectl) works as unprivileged user as well - Good for our setup
- install helm on ubuntu (client) as root: `snap install --classic helm`
 - this installs helm3
- Please only use: helm3. No server-side components needed (in cluster)
 - Get away from examples using helm2 (hint: `helm init`) - uses tiller

Simple Walkthrough (Example 0: Step 1)

```
## Repo hinzufügen
helm repo add bitnami https://charts.bitnami.com/bitnami
## gecachte Informationen aktualisieren
helm repo update

helm search repo bitnami
## helm install release-name bitnami/mysql
```

Simple Walkthrough (Example 0: Step 2: for learning - pull)

```
helm pull bitnami/mysql
tar xvfz mysql*
```

Simple Walkthrough (Example 0: Step 3: install)

```
helm install my-mysql bitnami/mysql
## Chart runterziehen ohne installieren
## helm pull bitnami/mysql

## Release anzeigen zu lassen
helm list

## Status einer Release / Achtung, heisst nicht unbedingt nicht, dass pod läuft
helm status my-mysql

## weitere release installieren
## helm install neuer-release-name bitnami/mysql
```

Under the hood

```
## Helm speichert Informationen über die Releases in den Secrets
kubectl get secrets | grep helm
```

Example 1: - To get know the structure

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update
helm pull bitnami/mysql
tar xzvf mysql-9.0.0.tgz

## Show how the template would look like being sent to kube-api-server
helm template bitnami/mysql
```

Example 2: We will setup mysql without persistent storage (not helpful in production ;o())

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo bitnami
helm repo update

helm install my-mysql bitnami/mysql
```

Example 2 - continue - fehlerbehebung

```
helm uninstall my-mysql
## Install with persistentStorage disabled - Setting a specific value
helm install my-mysql --set primary.persistence.enabled=false bitnami/mysql

## just as notice
## helm uninstall my-mysql
```

Example 2b: using a values file

```
## mkdir helm-mysql
## cd helm-mysql
## vi values.yml
primary:
  persistence:
    enabled: false
```

```
helm uninstall my-mysql
helm install my-mysql bitnami/mysql -f values.yml
```

Example 3: Install wordpress

Example 3.1: Setting values with --set

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress \
  --set wordpressUsername=admin \
  --set wordpressPassword=password \
  --set mariadb.auth.rootPassword=secretpassword \
  bitnami/wordpress
```

Example 3.2: Setting values with values.yml file

```
cd
mkdir -p manifests
cd manifests
mkdir helm-wordpress
cd helm-wordpress
nano values.yml
```

```
## values.yml
wordpressUsername: admin
wordpressPassword: password
mariadb:
  auth:
    rootPassword: secretpassword
```

```
## helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-wordpress -f values.yml bitnami/wordpress
```

Referenced

- <https://github.com/bitnami/charts/tree/master/bitnami/mysql/#installing-the-chart>
- <https://helm.sh/docs/intro/quickstart/>

Installation, Upgrade, Uninstall helm-Chart exercise - simple (mariadb-cloudpirates)

Schritt 1: install mariadb von cloudpirates

```
## Mini-Step 1: Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.1 --dry-run=server
```

```
## Mini-Step 2: Installieren
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.1
```

```
## Geht das denn auch ?
kubectl get pods
## sehr gut in helm v4
helm status my-mariadb
```

Schritt 2: Exercise: Upgrade to new version

Schritt 2.1 Default values (auf terminal) ausfindig machen

```
## Recherchiere wie die Werte gesetzt werden (artifacthub.io) oder verwende die
folgenden Befehle:
helm show values oci://registry-1.docker.io/cloudpirates/mariadb
helm show values oci://registry-1.docker.io/cloudpirates/mariadb | less
```

Schritt 2.2 Upgrade und resources ändern

```
cd
mkdir -p mariadb-values
cd mariadb-values
mkdir prod
cd prod
```

```
nano values.yaml
```

```
resources:
  limits:
    memory: 300Mi
  requests:
    memory: 300Mi
    cpu: 100m
```

```
cd ..
```

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.3 --dry-run -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.5.3 -f prod/values.yaml
```

```
kubectl get pods
```

Umschauen

```
kubectl get pods
## Ab Version 4 (helm) sinnvoll
helm status my-mariadb
helm list
## alle helm charts anzeigen, die im gesamten Cluster installiert wurden
helm list -A
helm history my-mariadb
```

Umschauen get

```
## Wo speichert er Information, die er später mit helm get abrufen kann
kubectl get secrets
```

```
helm get values my-mariadb
helm get manifest my-mariadb
## Zeile ausgeben und 4 Zeilen danach und 4 Zeilen davor
helm get manifest my-mariadb | grep "300Mi" -A4 -B4
## alles was ich ausgeben kann an Daten aus secrets .
helm get all my-mariadb
```

```
## Hack COMPUTED VALUES anzeigen lassen
## Welche Werte (values) hat er zur Installation verwendet
helm get all my-mariadb | grep -i computed -A 200
## besser Variante von David
helm get all my-mariadb | sed -n '/COMPUTED/, /HOOKS/p'
```

Tipp: values aus alter revision anzeigen

```
## Beispiel:
helm get values my-mariadb --revision 1
```

Schritt 3: Exercise: Upgrade to new version

Schritt 3.1. Upgrade und resources beibehalten

- Values wurden bereits im vorherigen Schritt angelegt

```
## Testen
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 --dry-run=server -f prod/values.yaml
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 -f prod/values.yaml
```

```
kubect1 get pods
## kein neuer pod
```

Schritt 3.2 Fehlgeschlagene Installation, wie lösen ?

```
## Schlägt fehle, weil mit dem upgrade bestimmte Felder nicht überschrieben dürfen,
die geändert wurden im Template
```

Lösung

- Deinstallieren (pvc bleibt erhalten auch beim Deinstallieren -> so macht das helm)
- Und wieder installieren in der neuen Version

```
## Frage, ist das pvc noch ?
kubect1 get pvc
## Ja !
```

NAME	STATUS	VOLUME	CAPACITY
data-my-mariadb-0	Bound	pvc-575fd652-e0da-46ed-b917-a2c28629e164	8Gi
ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
RWO	do-block-storage	<unset>	66m

```
## alte revisions behalten
helm uninstall my-mariadb --keep-history
kubect1 get pvc
## auch nach der Deinstallation ist der pvc noch da
## Super !!
```

```
## Real Upgrade
helm upgrade --install my-mariadb oci://registry-1.docker.io/cloudpirates/mariadb --
reset-values --version 0.10.1 -f prod/values.yaml
```

```
kubect1 get pods
helm get values my-mariadb
```

Änderung zwischen versionen (Warum trat der Fehler auf)

```
helm get manifest my-mariadb --revision 2 > rev2.yaml
helm get manifest my-mariadb --revision 3 > rev3.yaml
## schaut nach serviceaccount
diff rev2.yaml rev3.yaml
```

Uninstall

```
helm uninstall my-mariadb
## namespace wird nicht gelöscht
## händisch löschen
kubect1 delete ns <dein-name>
```

```
## crd's werden auch nicht gelöscht
kubectl create ns <dein-name>
```

Problem: OutOfMemory (OOM-Killer) if container passes limit in memory

- if memory of container is bigger than limit an OOM-Killer will be triggered
- How to fix. Use memory limit in the application too !
 - <https://techcommunity.microsoft.com/blog/appsonazureblog/unleashing-javascript-applications-a-guide-to-boosting-memory-limits-in-node-js/4080857>

Helm Spickzettel

Hilfe

```
helm help
helm help <command>
helm help upgrade
```

Alle helm-releases anzeigen

```
## im eigenen Namespace
helm list
## in allen Namespaces
helm list -A
## für einen speziellen
helm -n kube-system list
```

Helm - Chart installieren

```
## Empfehlung mit namespace
## Repo hinzufügen für Client
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install my-nginx bitnami/nginx --version 19.0.1 --create-namespace --
namespace=app-<namenskuerzel>
## besser upgrade --install --reset-values (immer alle Werte wie im aktuellen
Installationsbefehl
helm upgrade --install --reset-values my-nginx bitnami/nginx --version 19.0.1 --
create-namespace --namespace=app-<namenskuerzel>
```

Helm - prüfen (nach Installation)

```
helm status my-release
helm get values my-release
helm get manifest my-release
```

Helm - Suche

```
## welche Repos sind konfiguriert
helm repo list
```

```
helm search repo bitnami
helm search hub
```

Helm - template

```
## Rendern des Templates
helm repo add bitnami https://charts.bitnami.com/bitnami
helm template my-nginx bitnami/nginx
helm template bitnami/nginx
```

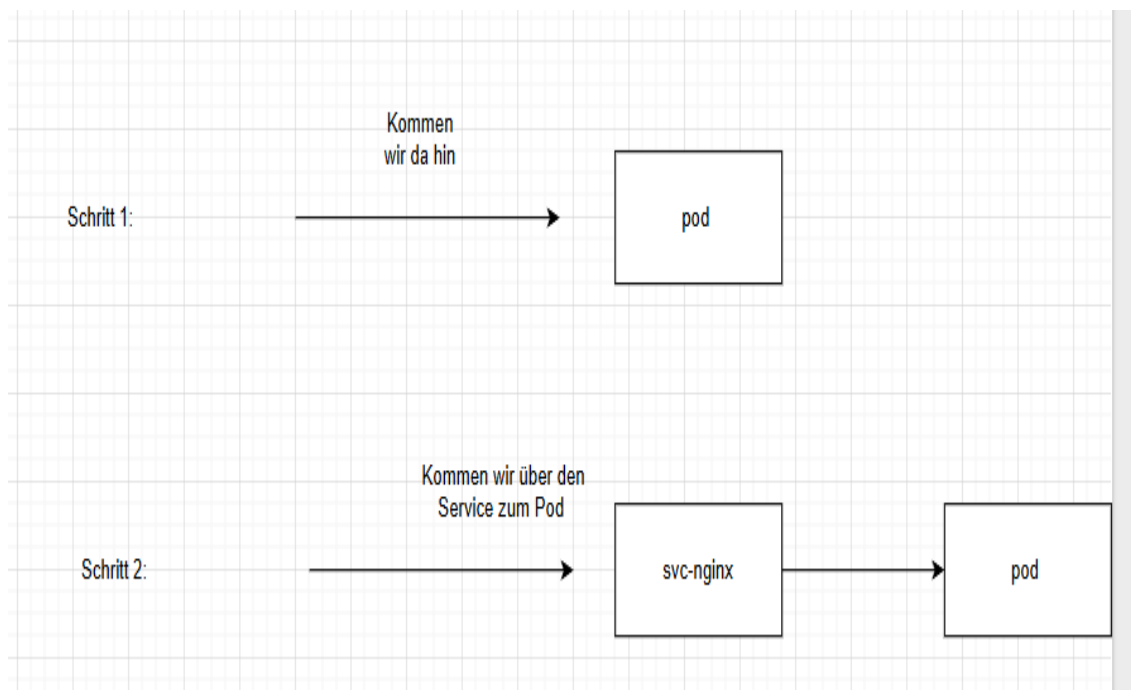
Kubernetes Debugging (Troubleshooting)

Netzwerkverbindung zum Pod testen

Situation

Managed Cluster und ich kann nicht auf einzelne Nodes per ssh zugreifen

Was wollen wir testen (auf der Verbindungsebene) ?



Behelf: Eigenen Pod starten mit busybox

```
## der einfachste Weg
kubectl run podtest --rm -it --image busybox
```

```
## Alternative
kubectl run podtest --rm -it --image busybox -- /bin/sh
```

Example test connection

```
## wget befehl zum Kopieren
ping -c4 10.244.0.99
wget -O - http://10.244.0.99
```

```
## -O -> Output (grosses O (buchstabe))
kubectl run podtest --rm -ti --image busybox -- /bin/sh
/ # wget -O - http://10.244.0.99
/ # exit
```

Befehle in pod ausführen - Übung

```
kubectl run my-nginx --image=nginx:1.23
```

```
kubectl exec my-nginx -- ls -la
kubectl exec -it my-nginx -- bash
kubectl exec -it my-nginx -- sh
```

```
## in der shell
cat /etc/os-release
cd /var/log/nginx
ls -la
exit
```

```
## Logs ausgeben
kubectl logs my-nginx
```

Welche Pods mit Namen gehören zu einem Service

```
kubectl get svc svc-nginx -o wide
kubectl get pods -l web=my-nginx
```

Übung: Service-Verbindungsprobleme debuggen mit kubectl debug (ohne NetworkPolicy)

Hintergrund

Wenn Pods sich gegenseitig nicht erreichen koennen, gibt es zwei haeufige Ursachen - beide zeigen `Connection refused`, aber aus unterschiedlichen Gruenden:

Fehlerbild	Ursache	Erkennungsmerkmal
Connection refused	Falscher Service-Selector - keine Endpoints	<code>kubectl get endpoints</code> zeigt <code><none></code>
Connection refused	Falscher targetPort - Endpoint zeigt falsche Port	<code>kubectl get endpoints</code> zeigt Endpoint mit falscher Port

`kubectl debug` schleust einen ephemeral Container mit Debug-Tools in einen laufenden Pod ein - ohne den Pod neu starten zu muessen.

Schritt 1: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir 21-debug-service
cd 21-debug-service
```

Schritt 2: Backend Deployment und Service anlegen

Achtung: Im Service steckt ein Fehler - den sollt ihr selbst finden.

```
nano 01-backend.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
        tier: backend
    spec:
      containers:
        - name: backend
          image: nginx:alpine
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: backend-api
  ports:
    - port: 80
      targetPort: 80
```

```
kubectl create ns debug-<dein-name>
kubectl apply -f 01-backend.yml -n debug-<dein-name>
```

Schritt 3: Frontend Deployment und Service anlegen

Das Frontend läuft als minimales Python-Image (kein curl, wget, nc) und startet einen einfachen HTTP-Server auf Port 8080.

Achtung: Auch im Frontend-Service steckt ein Fehler.

```
nano 02-frontend.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
        tier: frontend
    spec:
      containers:
      - name: frontend
        image: python:3.12-slim
        command: ["python", "-m", "http.server", "8080"]
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  selector:
    app: frontend-api
  ports:
  - port: 8080
    targetPort: 8080
```

```
kubectl apply -f 02-frontend.yml -n debug-<dein-name>
```

```
kubectl get pods -n debug-<dein-name>
kubectl get services -n debug-<dein-name>
```

Erwartete Ausgabe:

NAME	READY	STATUS	RESTARTS	AGE
backend-xxx	1/1	Running	0	30s
frontend-xxx	1/1	Running	0	20s

NAME	TYPE	CLUSTER-IP	PORT(S)
backend-svc	ClusterIP	10.x.x.x	80/TCP
frontend-svc	ClusterIP	10.x.x.x	8080/TCP

Problem 1: FE zu Backend - Falscher Selector, keine Endpoints

Schritt 4: kubectl debug - Verbindung testen

```
FE_POD=$(kubectl get pod -n debug-<dein-name> -l app=frontend -o
jsonpath='{.items[0].metadata.name}')
echo $FE_POD
```

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

Im Debug-Container:

```
nslookup backend-svc
wget -qO- http://backend-svc --timeout=5
```

Erwartete Ausgabe:

```
Name:      backend-svc.debug-<dein-name>.svc.cluster.local
Address: 10.x.x.x

wget: can't connect to remote host (10.x.x.x): Connection refused
```

DNS loest auf - aber Connection refused . Kein Listener hinter dem Service.

```
exit
```

Schritt 5: Endpoints und Selector pruefen

```
kubectl get endpoints backend-svc -n debug-<dein-name>
kubectl describe service backend-svc -n debug-<dein-name> | grep -E
'Selector|Port|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=backend --show-labels
```

Diagnose: Service sucht `app=backend-api` , Pods haben `app=backend` .

Schritt 6: Fix - Selector korrigieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
-p '{"spec":{"selector":{"app":"backend"}}}'
```

```
kubectl get endpoints backend-svc -n debug-<dein-name>
```

Erneut testen:

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
  --image=busybox:1.36 \
  --target=frontend \
  --profile=general \
  -- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: `<h1>Welcome to nginx!</h1>` - Verbindung OK.

Problem 2: FE zu Backend - Falscher targetPort

Schritt 7: targetPort kaputt konfigurieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
  -p '{"spec":{"ports":[{"port":80,"targetPort":8080}]}}'
```

Schritt 8: kubectl debug - Verbindung testen

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
  --image=busybox:1.36 \
  --target=frontend \
  --profile=general \
  -- sh
```

```
wget -qO- http://backend-svc --timeout=5
```

Erwartete Ausgabe:

```
wget: can't connect to remote host (10.x.x.x): Connection refused
```

Wieder `Connection refused` - aber diesmal aus anderem Grund.

```
exit
```

Schritt 9: Diagnose - Endpoint vorhanden aber Port falsch

```
kubectl get endpoints backend-svc -n debug-<dein-name>
kubectl describe service backend-svc -n debug-<dein-name> | grep -E
'Port|Target|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=backend -o
jsonpath='{.items[0].spec.containers[0].ports}'
```

Diagnose: TargetPort zeigt auf 8080, Container lauscht auf 80.

Schritt 10: Fix - targetPort korrigieren

```
kubectl patch service backend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":80,"targetPort":80}]}}'
```

```
kubectl debug -it $FE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=frontend \
--profile=general \
-- sh
```

```
wget -qO- http://backend-svc --timeout=5
exit
```

Erwartete Ausgabe: <h1>Welcome to nginx!</h1> - Verbindung OK.

Problem 3: Rueckweg Backend zu Frontend - Falscher Selector

Schritt 11: kubectl debug auf Backend-Pod - Verbindung zum Frontend testen

```
BE_POD=$(kubectl get pod -n debug-<dein-name> -l app=backend -o
jsonpath='{.items[0].metadata.name}')
echo $BE_POD
```

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

Im Debug-Container:

```
nslookup frontend-svc
wget -qO- http://frontend-svc:8080 --timeout=5
```

Erwartete Ausgabe:

```
Name:      frontend-svc.debug-<dein-name>.svc.cluster.local
Address: 10.x.x.x

wget: can't connect to remote host (10.x.x.x): Connection refused
```

```
exit
```

Schritt 12: Endpoints und Selector pruefen

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
kubectl describe service frontend-svc -n debug-<dein-name> | grep -E
'Selector|Port|Endpoint'
kubectl get pods -n debug-<dein-name> -l app=frontend --show-labels
```

Diagnose: Service sucht `app=frontend-api` , Pods haben `app=frontend` .

Schritt 13: Fix - Selector korrigieren

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"selector":{"app":"frontend"}}}'
```

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
```

Erneut testen:

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwartete Ausgabe: <title>Directory listing for /</title> - Verbindung OK.

Problem 4: Rueckweg Backend zu Frontend - Falscher targetPort

Schritt 14: targetPort kaputt konfigurieren

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":8080,"targetPort":9090}]}}'
```

Schritt 15: kubectl debug - Verbindung testen

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
```

Erwartete Ausgabe:

```
wget: can't connect to remote host (10.x.x.x): Connection refused
```

```
exit
```

Schritt 16: Diagnose und Fix

```
kubectl get endpoints frontend-svc -n debug-<dein-name>
kubectl describe service frontend-svc -n debug-<dein-name> | grep -E
'Port|Target|Endpoint'
```

```
kubectl get pods -n debug-<dein-name> -l app=frontend -o
jsonpath='{.items[0].spec.containers[0].ports}'
```

Diagnose: TargetPort zeigt auf 9090, Container lauscht auf 8080.

```
kubectl patch service frontend-svc -n debug-<dein-name> \
-p '{"spec":{"ports":[{"port":8080,"targetPort":8080}]}}'
```

```
kubectl debug -it $BE_POD -n debug-<dein-name> \
--image=busybox:1.36 \
--target=backend \
--profile=general \
-- sh
```

```
wget -qO- http://frontend-svc:8080 --timeout=5
exit
```

Erwartete Ausgabe: <title>Directory listing for /</title> - Rueckweg OK.

Aufräumen

```
kubectl delete namespace debug-<dein-name>
```

Zusammenfassung

Problem	Richtung	kubectl get endpoints	Diagnose	Fix
Falscher Selector	FE → Backend	<none>	Selector passt nicht zu Pod-Labels	Selector anpassen
Falscher targetPort	FE → Backend	Port falsch	TargetPort != ContainerPort	targetPort anpassen
Falscher Selector	Backend → FE	<none>	Selector passt nicht zu Pod-Labels	Selector anpassen
Falscher targetPort	Backend → FE	Port falsch	TargetPort != ContainerPort	targetPort anpassen

Merkhilfe: Endpoints leer → Selector-Problem. Endpoints vorhanden aber falsche Port → targetPort-Problem.